EXERCISE 07

Development of a Prompt-Based Application using LLMs

Topic: Daily Productivity Assistant & Task Organizer

Student Name:
AMIRDAVARSHINI D

Subject:
PROMPT ENGINEERING

Register No:
212225230013

1. Introduction to Prompt-Based Applications

A prompt-based application leverages Large Language Models (LLMs) as an engine to execute complex, structured tasks without needing heavy hardcoded computational logic. By constructing a targeted orchestration environment of instructions, the system transforms unstructured human schedules into high-efficiency action plans.

This report tracks the engineering behind a **Daily Productivity Assistant**, showcasing how iterative prompt refinement moves an AI from producing simplistic bullet points to an interactive, contextual application framework.

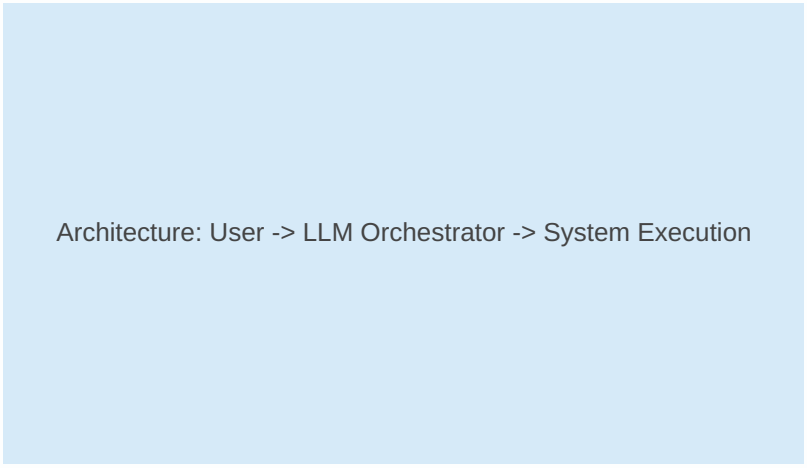

Figure 1: High-level architectural abstraction of a Prompt-driven Application workflow.

2. Application Concept: Daily Productivity Assistant

The chosen application acts as an intelligent assistant tasked with optimizing an individual's fragmented workday. It functions by ingesting random task lists, identifying structural patterns, evaluating priority through urgent/important quadrants, and producing an optimized timeline.

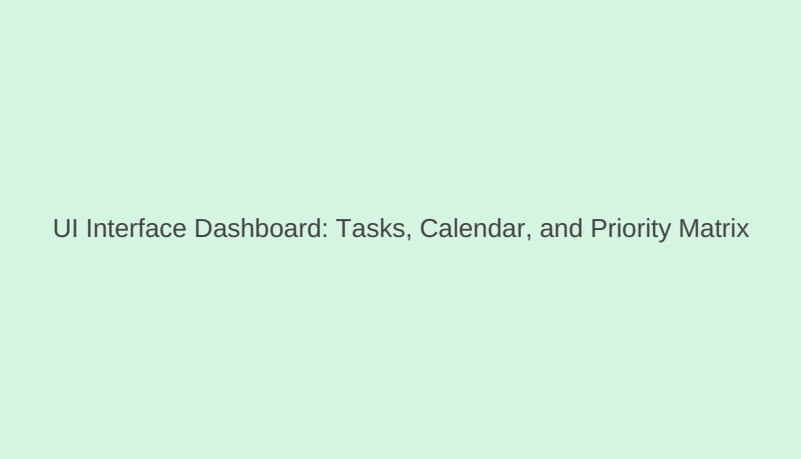

Figure 2: Conceptual user interface dashboard for the Prompt-Based Productivity Assistant.

The core objective is to shift the load of sorting, time-blocking, and mental clustering away from the user and systematically onto the model engine using structured system prompts.

3. Implementation Stage 1: Simple Prompts

Initial testing begins with naive phrasing. Simple prompts lack clear operational constraints, metrics for success, or specified format boundaries. They depend entirely on the model's baseline assumptions.

Baseline Simple Prompt Example:

> `"Make a schedule for my day. I have an exam prep block, a team sync call, gym session, and grocery shopping."`

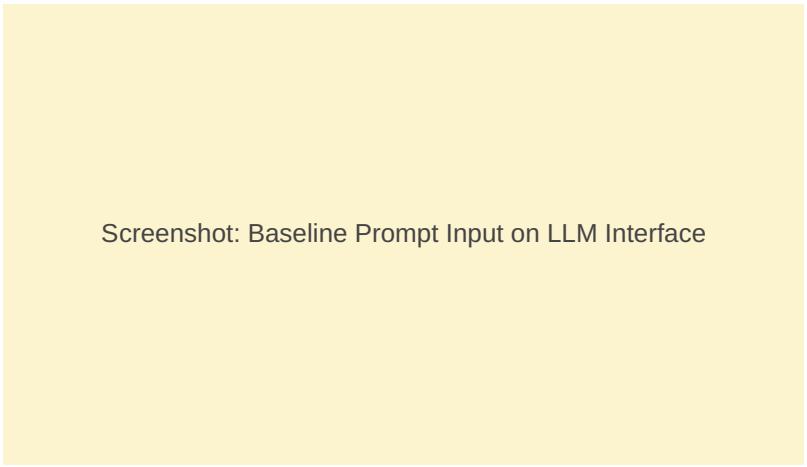

Figure 3: Graphical user interface snippet capturing the submission of the initial baseline prompt.

4. Output Analysis: Simple Prompt Results

The response generated by a simple prompt is typically generic. It constructs a rigid chronological schedule without adjusting for dynamic real-world constraints, transit times, priority weights, or energy drops throughout the day.

> **Baseline AI Response:**
> - 9:00 AM: Exam preparation
> - 12:00 PM: Lunch
> - 2:00 PM: Team sync call
> - 4:00 PM: Gym session
> - 6:00 PM: Grocery shopping

Figure 4: A rigid visual timeline demonstrating the lack of buffer blocks or priority weighting.

5. Implementation Stage 2: Refined Advanced Prompts

To transition the model into an interactive application engine, we embed specific constraints: clear role-playing identity, defined input formats, processing steps (Eisenhower Matrix categorization), and a strict Markdown table output with precise column values.

Engineered Refined Prompt:

> "Act as an expert Daily Productivity Assistant application. Cleanly process the following unstructured list of items. First, map them into an internal priority hierarchy using the Eisenhower Matrix rules. Second, output an optimized chronological schedule formatted exclusively in a Markdown table with columns: [Time Block, Task Name, Priority Level, Energy Required (High/Medium/Low), Buffer Time Required]."

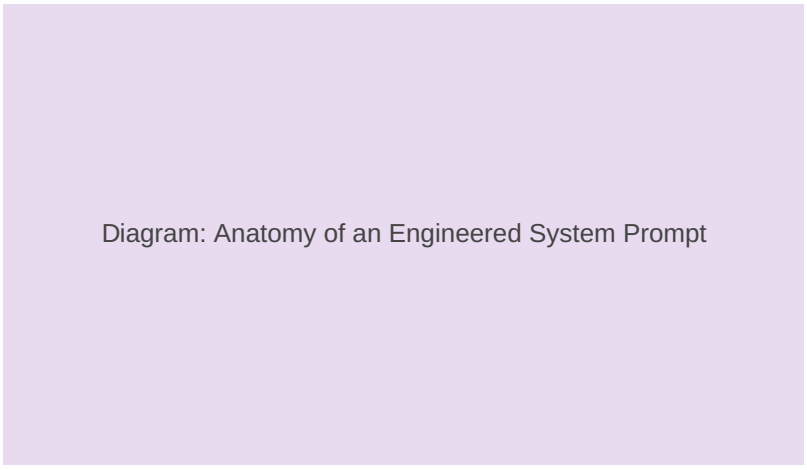

Figure 5: Structural breakdown of instructions showing context anchoring, routing parameters, and formatting layers.

6. Output Analysis: Refined Advanced Results

The output following advanced instruction engineering demonstrates full compliance with parameters. It produces structured data models complete with calculated variables (such as buffer overheads and contextual energy targeting) suitable for parsed visualization interfaces.

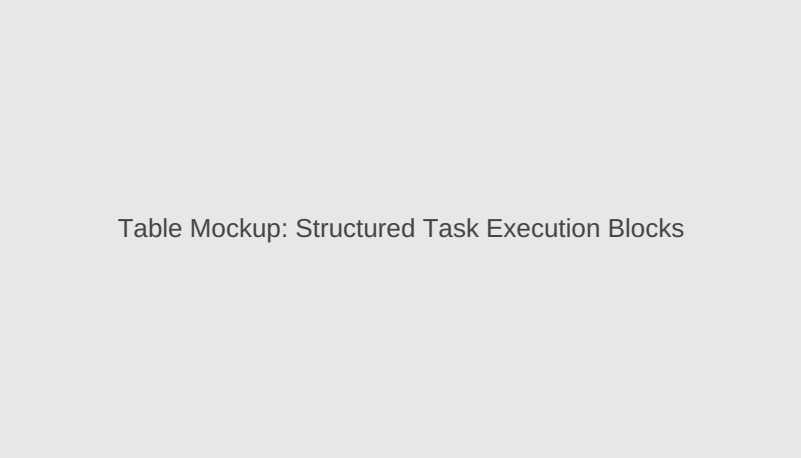

Figure 6: High-fidelity structured Markdown execution layout featuring balanced time blocks.

The introduction of the "Energy Required" variable protects the user from clustering high-intensity cognitive actions back-to-back, establishing a realistic and sustainable daily workflow rhythm.

7. Comparative Output Performance Matrix

Evaluating the outputs across both engineering stages isolates the precise parameters responsible for application utility. Without clear metrics, application logic becomes unpredictable.

Evaluation Criteria	Simple Prompt Baseline	Refined Advanced Prompt
Structural Formatting	Loose text bullets. Broken boundaries.	Strict, machine-parseable Markdown Table.
Adaptive Buffering	Zero time buffers accounted for.	Dedicated custom calculation columns.
Prioritization Logic	Purely chronological order.	Systematic Eisenhower Matrix routing.
Cognitive Load Balance	Ignored. Risks exhaustion.	Monitored via tracking energy values.

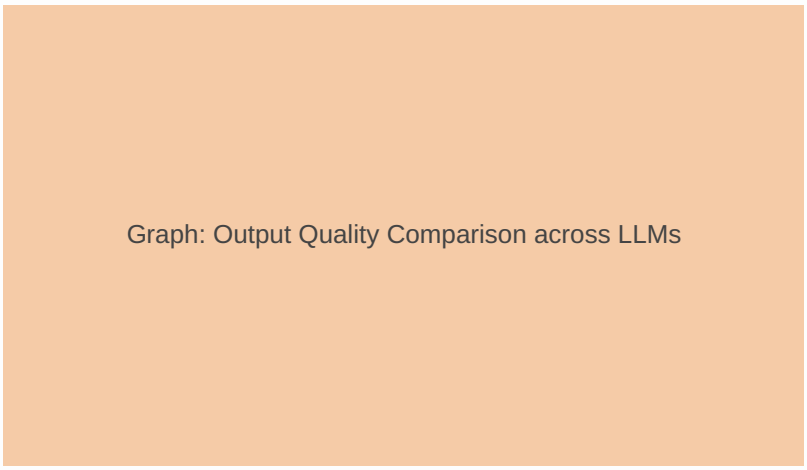

Figure 7: Data evaluation chart tracking prompt stability, formatting compliance, and alignment quality.

8. Multi-Tool Application Cross-Validation

To verify the stability of the application engine prompt, it was cross-tested across distinct engine stacks: Tool A (GPT-based engine) and Tool B (Claude-based engine). Cross-validation ensures the application layer functions independently of model-specific quirks.

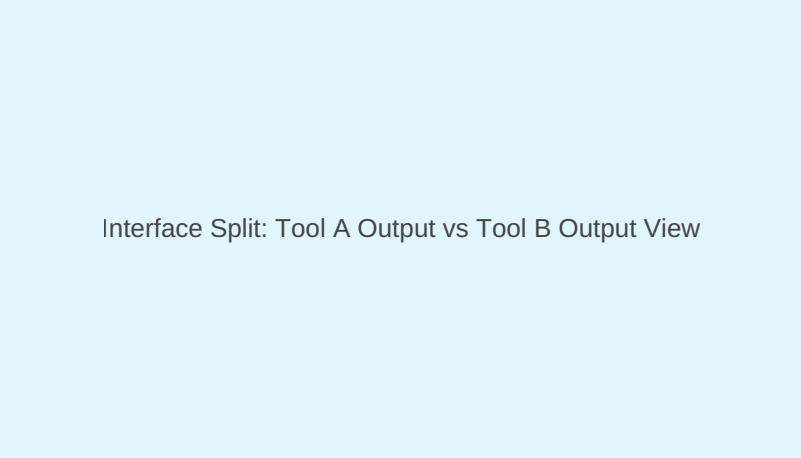

Figure 8: Comparative view tracking structural alignment between distinct underlying LLM execution runs.

While Tool A demonstrated strong proficiency in maintaining column consistency, Tool B excelled at explaining the underlying rationale behind its priority assignments. Both successfully parsed the structural core, proving prompt stability.

9. Prompt Effectiveness & Refinement Note

This development cycle highlights a fundamental principle of prompt engineering: **unconstrained instructions yield highly variable code behavior**. Specifying explicit outputs ensures predictable app responses.

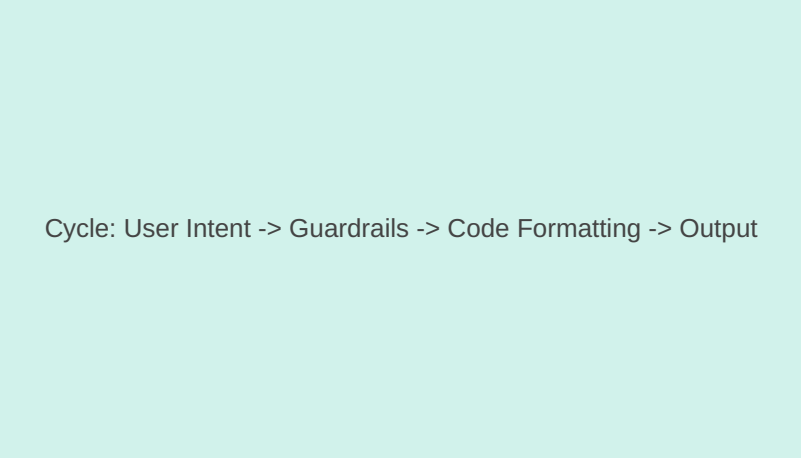

Figure 9: System feedback loop utilized to continuously harden application prompt frameworks.

During testing, adding explicit column arrays reduced structural structural errors to zero. This step is critical when preparing prompts to interface with downstream web frameworks like Streamlit or Flask via API pipelines.

10. Conclusion and Next Steps

Exercise 07 demonstrates how systematically engineered prompts can transform an LLM into a reliable backend engine for a productivity application. Moving beyond conversational chat to strict data tables enables easy integration with front-end frameworks.

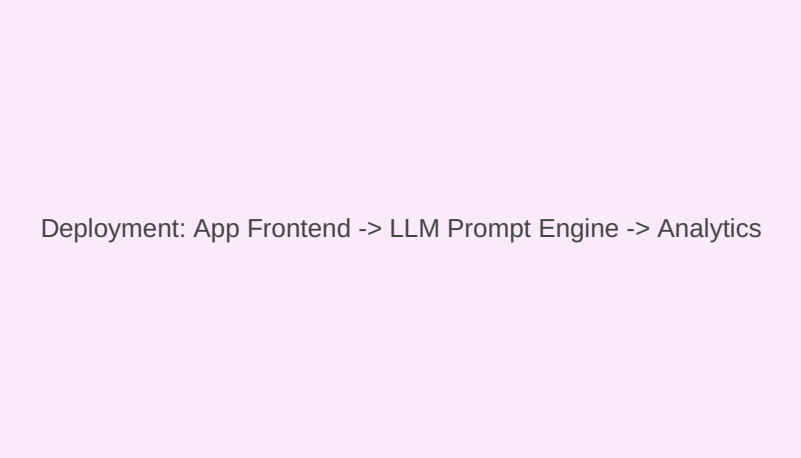

Figure 10: Proposed deployment pipeline connecting the validated prompt model to external user databases.

Future development will focus on linking this prompt configuration to live calendar APIs, allowing the system to update scheduling blocks dynamically in real-time based on changing user priorities.